ESPRIT School of Engineering

Advanced Data Science Project · Final assessment

CallVerse

A multi-agent digital twin of an
e-commerce customer service centre

Technical report

Group members	Hamza Farhani Jihed Hmila Abdellaoui Woujoud Mohamed Aziz Kadri Mohamed Yessine El Ichi
Class	5DS3
Tutor	[Tutor's name]
Date	11 November 2026

Academic year 2026–2027

Abstract

A customer service centre fails in two expensive ways: too few advisors when contacts spike, and too many when it is quiet. Decisions about staffing and routing are usually tested directly on real customers. **CallVerse** is a *digital twin* of an e-commerce support centre: a virtual copy in which a manager can test a decision before applying it. It forecasts contacts per reason from real order events (Olist, 99,441 orders), replays them in a discrete-event simulation, and lets four cooperating agents act inside it: a *Client* agent plays customers, a *Workforce* agent routes chats and plans shifts, an *Advisor* agent answers with a retrieval-augmented language model and business tools, and a *Quality* agent grades every conversation. Each of our four data science objectives has a simple baseline to beat, a backup, and a switch rule fixed in advance. The forecast reduces the error of the naive baseline from 31.6% to **25.3%** wMAPE, and the twin passes every fidelity check. Shift planning with constraint programming cuts labour cost by **40%** while 92.4% of chats are still answered within 20 seconds (target 80%). Testing a plan in the twin before a peak week cuts understaffed hours by **60–84%**. Reinforcement-learning routing reduces the speed of answer by 13.9% on unseen stressed days but is not robust in full peak weeks, so a tuned rule stays in production. Semantic retrieval raises recall@4 from 80.4% to **96.9%**; the advisor that lets a rules engine decide and the language model write gives 12/12 verified policy-correct actions, while the automatic judge is not yet trusted (kappa below 0.4). The application (two web apps, an API, an MCP server and a live contact-centre view) ships as one Docker container, with 200 automated tests and one command to reproduce every result.

Contents

1 Introduction and problem statement **2**

2 Data **3**

3 Methodology and architecture **5**

4 Experiments **8**

5 Results **9**

6 Critical analysis and limitations **15**

7 Deployment and reproducibility **16**

8 Conclusion and future work **19**

9 Declaration of the use of generative AI tools **19**

1 Introduction and problem statement

1.1 Context: an e-commerce customer service centre

Most contacts to an online shop's customer service are caused by orders: a parcel is late ("where is my order?"), a customer wants to cancel or get a refund, an item arrives damaged or wrong, or an invoice or address must be changed. These causes are largely *predictable*: the number of orders, the promotion calendar and the carriers' delays are known days in advance. Yet many centres still plan from last year's averages and react to peaks when they happen. This status quo has four costs:

- **Peaks hurt customers**: long waits, abandoned chats, repeat contacts and bad reviews.
- **Quiet periods waste money**: staffing is the largest cost of a contact centre, and paid advisors sit idle.
- **Decisions are tested on real customers**: there is no safe place to ask "what if a promotion doubles the tracking questions?" or "what if 20% of the advisors are sick?".
- **AI assistants are adopted without measurement**: chatbots answer customers, but few companies check whether the answers follow the company policy, or whether an automatic grader agrees with humans.

1.2 Our answer: a digital twin with four agents

CallVerse is a *digital twin* of such a centre: a virtual copy that behaves like the real one, so that a decision can be tested in seconds before customers feel it. The twin is driven by real e-commerce data (the Olist marketplace) and is used by four cooperating *agents*, programs that observe a situation and act on it:

- the **Client agent** plays customers, each with a real order, a problem and a personality;
- the **Workforce agent** recognises each customer's problem, sends the chat to the right advisor, and plans the advisors' shifts;
- the **Advisor agent** answers, using the company's policy documents and business tools (order lookup, refund eligibility, tickets);
- the **Quality agent** grades every conversation on a five-criterion scorecard.

1.3 Business and data science objectives

We framed the project as four *business objectives* (BO), each measured by business KPIs, and four *data science objectives* (DSO), each measured by a model metric (Table 1). A KPI tells whether the centre is doing better; a DSO metric tells whether the AI component works. Three rules apply to every objective: (1) the main idea must beat a *simple baseline*; (2) a *backup* covers the whole objective with less effort; (3) a *switch rule*, fixed before the experiments, decides whether the main idea or the backup goes to production. DSO4 is the foundation: DSO1 and DSO3 are trained and tested inside its simulation.

What success means. The centre's service target is the industry's "80/20" rule: 80% of chats answered within 20 seconds. A good plan meets it at the lowest labour cost and respects labour law (at most 5 days and 40 hours a week, at least 11 hours of rest between two shifts). A good answer applies the company policy (for example: a parcel more than 7 days late triggers a carrier investigation) and leaves the customer satisfied. Every claimed gain must be measured against its baseline on the same simulated customers, with a 95% confidence interval.

Table 1: From business objectives to data science objectives. "Rule" is the switch rule decided in advance.

Business objective	DSO and main idea	Baseline	Backup	Rule
BO1 Customers wait less	DSO1: real-time routing learned by reinforcement learning (MaskablePPO)	First come, first served (FIFO) with skill matching	Priority score with tuned weights	≥10% less waiting in the twin's peak weeks
BO2 Problems solved correctly, higher satisfaction	DSO2: advisor with retrieval-augmented generation and tools, plus an LLM judge	Keyword FAQ search; templates	Intent classifier + approved templates + checklist reviewer	≤10% wrong answers; judge–human kappa ≥ 0.4
BO3 Lower cost, same service	DSO3: forecast + constraint-programming shift planning (CP-SAT)	Fixed schedule sized for the peak	Erlang C + shift templates + overtime rule	≥5% cheaper while meeting the service target and the labour rules
BO4 Prepare for peaks by testing first	DSO4: contact forecast per reason + discrete-event simulation	Same day last week; Erlang C queueing formula	Statistical forecast + queueing model	Beat the naive forecast; the twin passes its fidelity checks

1.4 Contributions and structure of the report

We deliver (1) a reproducible data pipeline from raw Olist and Bitext files to a relational store and a knowledge base; (2) a forecast and a validated simulation engine; (3) four agents combining machine learning, reinforcement learning, optimisation and language models; (4) an evaluation framework with paired experiments, a scorecard and live monitoring of every AI component; and (5) a deployed application. Section 2 presents the data, Section 3 the methods and architecture, Section 4 the experimental protocol and Section 5 the results. Section 6 discusses limitations, Section 7 deployment and reproducibility, and Section 9 declares our use of generative AI tools.

2 Data

2.1 Sources

No public contact-centre logs exist, so we combine two public datasets with real behaviour and a documented generation step (Table 2).

- **Olist Brazilian e-commerce dataset** [17]: 99,441 real, anonymised orders placed between 4 September 2016 and 17 October 2018, with items, payments, reviews, customers, sellers, products and every delivery timestamp (purchase, approval, carrier hand-off, delivery, estimate). Licence CC BY-NC-SA 4.0.
- **Bitext retail e-commerce customer-service dataset** [3]: 44,884 customer messages labelled with 46 intents (a customer's goal, such as `track_order`). We keep the **17 intents linked to an order**, grouped into four families: *where is my order* (wismo, 4 intents), *cancel/refund* (3), *after delivery* (7) and *order admin* (3). Licence CDLA-Sharing 1.0.
- **Knowledge base**: 10 policy documents written for the project (refunds, returns, delivery delays, payments, invoices, an advisor playbook...), split into 58 sections.

Table 2: Data used by CallVerse (SQLite store and processed tables).

Table	Rows	Origin and use
Orders / customers	99,441	Olist; order situation, delivery dates, the advisor's order lookup
Order items / payments	112,650 / 103,886	Olist; refund amounts, payment anomalies
Products / sellers	32,951 / 3,095	Olist; product and seller context
Reviews	99,224	Olist; satisfaction signal during exploration
Contacts / tickets	9,179 / 9,179	Generated from the orders' situations (Section 2.4)
Chat messages	60,837	Generated, with the wording of real Bitext messages
Knowledge-base sections	58	10 policy documents, embedded with bge-m3

2.2 Exploration and quality validation

Exploration (notebooks 01–02) confirmed that the orders are suitable: 97.0% are delivered, 1.1% shipped, 0.6% cancelled and 0.6% unavailable. All orders have a purchase time and an estimated delivery date; the actual delivery time is missing for 2,965 orders, the carrier hand-off for 1,783 and the approval for 160, which matches their statuses (an undelivered order has no delivery time). Review comments are often empty (58.7% have no message), and 610 products (1.9%) have no category.

The quality validation found five kinds of contradictions (Table 3). One correction matters for the whole project: the estimated delivery dates are stored at midnight, so comparing them with a delivery *timestamp* counted orders delivered later on the same day as "late". Measured by calendar date, **6,535 of 96,476 delivered orders (6.8%) are late**, not the 7,826 reported in a first version.

Table 3: Data quality findings. Each anomaly becomes a Boolean flag column; no row is deleted or imputed.

Finding (flag column)	Orders
Carrier hand-off before payment approval	1,359
Delivery before carrier hand-off	23
Review created before the purchase date (40 of them reuse a review id)	64
Order with more than one review	547
Payment total differs from items + freight by more than 1 BRL	249

2.3 Cleaning and feature preparation

Cleaning (notebooks 03–04) parses types and *flags* anomalies rather than deleting them, so record counts are preserved and no value is invented by imputation. It produces an **analytical order table of 77 features** (delivery delay, lateness, payment anomalies, review score, product and seller context...) and a daily activity table (orders purchased, handed to carriers, delivered, delivered late). Six tables that later layers depend on are protected by *data contracts*: Pandera schemas [2] check keys, types, allowed values and ranges (`callverse validate`); all pass.

2.4 Synthetic contacts, tickets and chats

Real contact logs are private, so contacts are generated from the real orders with explicit, documented assumptions (`configs/data.yaml`, notebooks 05–08 and 12):

1. Each order is placed in a *situation*: never delivered and overdue, delivered late, cancelled, problem after delivery, payment anomaly, or uneventful.

2. Each situation has a *contact probability* (for example, 35% of late deliveries contact support), an *intent mix*, and a *timing rule* relative to an order event (a delay-related contact comes after the estimated date has passed). Contact hours follow the hour-of-day profile of Olist purchases, and a promotion calendar adds known retail events.
3. The customer's opening message is a real Bitext message of the chosen intent.

The result is **9,179 contacts** from January 2017 to August 2018. We cross-checked the volume against a public benchmark ("where is my order" contacts at 3–10% of orders). Each contact then becomes a ticket with an outcome decided by the company rules (resolved in the chat 53.4%, follow-up needed 43.4%, escalated 3.2%), a handle time drawn from the simulation's assumptions, and a satisfaction survey answered 30% of the time (mean 3.93/5). The **60,837 chat messages** use approved answer templates. All generated records carry an `is_synthetic` flag. Generation is seeded, so it is reproducible.

2.5 Ethics, licences and confidentiality

Olist identifiers are already anonymised, and no personal data is collected. Synthetic records are flagged and the rates behind them are written in one configuration file, so they can be replaced by a client's real logs without code changes. We respect the licences: non-commercial, share-alike use of Olist (CC BY-NC-SA 4.0) and Bitext (CDLA-Sharing 1.0). The advisor never asks for card numbers or passwords (a prompt rule and a checklist item), customers are told they talk to an AI, and staffing recommendations remain human decisions. For the public deployment, a script removes the customers we created while testing (our own names) and the transcripts of our test chats before anything leaves the laptop (Section 7).

3 Methodology and architecture

3.1 Process and architecture

We followed CRISP-DM [4]: business understanding (the BO/DSO framework), data understanding and preparation (Section 2), modelling (forecast, twin, agents), evaluation (Section 4) and deployment (Section 7). The code is organised in five layers (Figure 1). A lower layer never imports a higher one; six `import-linter` contracts enforce this in the test suite.

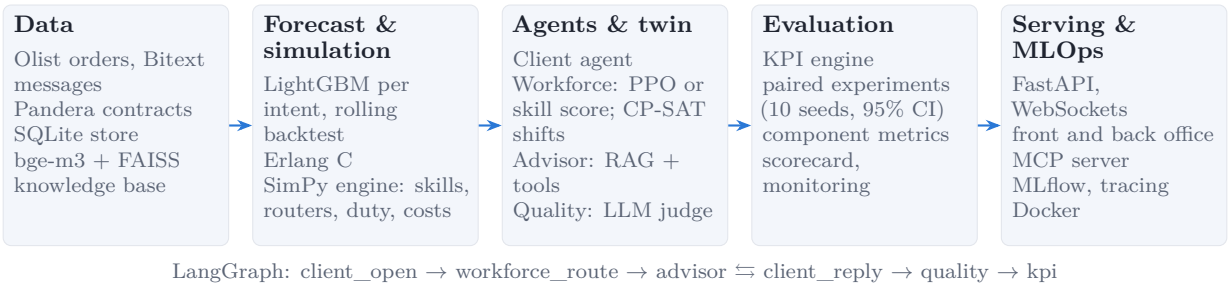


Figure 1: The five layers of CallVerse. Arrows show the only allowed direction of dependencies.

3.2 DSO4, part 1: forecasting the contacts

Target. Daily contacts per intent family (four series) and their total, 1 to 7 days ahead. **Features.** Every feature is lagged by at least 7 days, so the model never sees the future (a unit test checks it): the series' own history (lags 7, 14, 21 and 28 days, same-weekday mean, 7- and 28-day means), *order drivers* (orders purchased, handed to carriers, delivered and delivered late, orders promised for the day, overdue backlog) and the calendar (weekday, holidays, promotion windows, days since

the last major sale). **Models.** The DSO baseline "same day last week" (seasonal naive), a 4-week weekday mean, SARIMA$(1,1,1)(0,1,1)_7$ [10], Prophet with holidays and promotions [23], and one global LightGBM [12] with a Poisson objective and the family as a categorical feature. The error measure is the weighted mean absolute percentage error, which stays defined on days with few contacts:

$$\text{wMAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t y_t}. \tag{1}$$

3.3 DSO4, part 2: the digital twin

The twin is a discrete-event simulation written with SimPy [24]. Customers arrive as a non-homogeneous Poisson process: the daily volume (history or forecast) is spread over the hours with the observed hourly profile, and scaled ×25 to represent a mid-size centre. Each customer has an intent family, a lognormal handle time (means 3, 4, 6 and 7 minutes by family, coefficient of variation 0.6) and a persona that sets its patience (impatient 25%, normal 55%, patient 20%; mean patience 90, 180 and 360 s). Advisors have skills: 0.75× the handle time on their main family, 1.3× on a weak one. A duty matrix says who works each hour (regular or overtime) and a *router* decides who serves whom. Chats that arrive while the centre is closed (22:00–08:00) wait as an overnight backlog. Costs are €12 per hour, overtime ×1.5, and €8 per abandoned customer. The engine has two modes: a full run for evaluation, and a step-by-step mode that pauses at every routing decision for reinforcement learning.

Validation against queueing theory. With one skill and FIFO routing, the twin must reproduce the *Erlang C* model [8], which gives the probability that a customer waits when c advisors serve an offered load $A = \lambda \cdot \text{AHT}$:

$$P_{\text{wait}} = \frac{\frac{A^c}{c!}\frac{c}{c-A}}{\sum_{k=0}^{c-1}\frac{A^k}{k!} + \frac{A^c}{c!}\frac{c}{c-A}}, \qquad \text{SL}(t) = 1 - P_{\text{wait}}\, e^{-(c-A)\,t/\text{AHT}}. \tag{2}$$

Scenarios change a real week of history: a promotion spike (all families ×1.4–1.8), delivery delays (wismo ×2.5, handle time +10%), a staff shortage (20% absent), and the real busiest week after Black Friday 2017 (18 December 2017, about three times a normal November week, no multiplier).

3.4 DSO3: shift and overtime planning

From the forecast, Erlang C gives the number of advisors needed each hour to answer 80% of chats within 20 s with an occupancy of at most 90%. A two-stage model solved with OR-Tools CP-SAT [19] turns this requirement into a roster. *Stage 1, cheapest coverage*: integer variables $x_{d,t}$ (advisors on shift template t on day d; 27 templates of 4, 6 or 8 hours inside 08:00–22:00), $o_{d,h}$ (overtime advisors, at most 4 per hour and 60 hours per week) and $s_{d,h}$ (shortfall), with

$$\min \sum 12\, x\, \ell_t + 18\, o + 200\, s \quad \text{s.t.} \quad \sum_{t \ni h} x_{d,t} + o_{d,h} + s_{d,h} \geq r_{d,h} \;\; \forall d, h, \tag{3}$$

where ℓ_t is the template length and $r_{d,h}$ the requirement. *Stage 2, legal roster*: Boolean variables assign the chosen shifts to named advisors with one shift per day, at most 5 days and 40 hours per week, and at least 11 hours of rest; the headcount grows until a solution exists. Both stages solve to optimality in under a second; the solver runs on one thread so that the roster is reproducible.

3.5 DSO1: real-time routing

Routing is a sequential decision problem, written as a Markov decision process. *Decision point*: an advisor becomes free while at least two customers wait. *State*: the 10 oldest waiting customers (time waited, family, persona, live chat or overnight message, expected handle time with *this* advisor), the queue length, the backlog, free and on-duty advisors and the time of day. *Action*: which

waiting customer to serve; empty slots are masked. *Reward*: minus the customer-seconds of waiting since the last decision (overnight messages weighted 0.1), minus 600 per abandonment. By Little's law [16], minimising total waiting minimises the mean wait. We train MaskablePPO [22, 9] (Stable-Baselines3 [20], Gymnasium [25]) on single days of February 2017–March 2018 made busier than planned (load ×1.0–1.8, each family surging ×1.0–1.6, 5–30% absences): 2,394 episodes, 300,000 decisions, a 128×128 network. **Backup**: a priority score w_1 wait + w_2 skill match + w_3 short job + w_4 impatient, whose weights are tuned by random search in the twin (best: 0.19, 1.80, 2.69, 0.08).

3.6 DSO2: the language agents

Workforce agent (routing a chat). A TF-IDF + logistic-regression classifier [18] predicts the intent of the first message (99.0% accuracy, macro-F1 0.991 on 1,602 held-out Bitext messages) and sends the chat to an advisor of that family, with the expected wait from Erlang C.

Advisor agent. The loop is: understand the request, retrieve the relevant policy, call tools, write the reply. *Retrieval*: bge-m3 [5] embeddings (multilingual, 1,024 dimensions) of the 58 knowledge-base sections in an exact FAISS index [11]; the top 3 sections are kept, with a "no answer" threshold at a cosine of 0.57. This is retrieval-augmented generation (RAG) [15]: the model writes from retrieved evidence instead of from memory. *Tools* work on the SQLite store, "as of" the moment the customer writes: `lookup_order`, `get_ticket_history`, `check_refund_eligibility`, `open_ticket`, `search_knowledge_base` and the company rules engine `recommend_resolution`. We compare four variants:

- **template** (backup): the classifier's intent selects an approved answer filled with order facts;
- **rag** (pure idea A): the LLM decides which tools to call and writes freely;
- **rag_policy** (hybrid): once the order number is known, the pipeline itself calls `lookup_order` and the rules engine; the LLM receives the decision and only *writes* it;
- **agentic**: the hybrid plus tools the LLM chooses itself, in a ReAct-style loop [26]: a reflection step that fact-checks the draft against the data (`check_draft_reply`), a hand-off to a human (`escalate_to_human`), a check of bank codes against a public list (`validate_bank_code`) and a memory of earlier contacts (`get_customer_history`).

The first LLM runs exposed failure modes that we turned into guardrails: the model called `lookup_order` with invented ids (9 of 20 calls failed), so order tools are only offered once an order number appears in the chat (*tool gating*); it sometimes answered before checking the order, so a reply given before verification always ends with the order-number request. LLM-chosen tool calls go through our **Model Context Protocol** (MCP) [1] server, which outside applications can also use.

Client agent. It plays the customer of a real contact (opening message, persona, knowledge of its order) and ends with a 1–5 satisfaction score. It has three tools of its own: `view_my_order_page` (to check a status or date the advisor states), `check_my_waiting_time` (its patience budget) and `send_evidence` (photos or a payment statement, built from the real order, never written by the LLM).

Quality agent. A scorecard of five criteria rated 1–5 (compliance, relevance, satisfaction, sentiment, resolution), filled by an LLM judge [28] from a *different model family* than the advisor, to avoid a model grading its own style. The judge receives the policy-correct reference. A rule-based checklist is the offline backup. Correctness is also measured *deterministically*: the action chosen by the advisor must equal the action the rules give for the true intent.

The agents are chained with LangGraph [13], whose state (messages, routing decision, reference, scorecard, KPIs) is streamed step by step to the user interface.

3.7 LLM setup: three tiers

Every agent works at three levels, so the twin never stops:

1. **Hosted open models through OpenRouter (main, since 3 November 2026)**, free tier: NVIDIA Nemotron 3.5 Super for the advisor, Nemotron 3.5 Lightning for the client, and Xing 4.0 (another family) for the judge, each with a fallback model. Reasoning mode is switched off: these models otherwise returned empty answers. The free tier allows 20 requests per minute and 50 per day; when the daily quota is used, OpenRouter is skipped until 00:00 UTC.
2. **Local Hugging Face models (backup)**: SmolLM3-3B (advisor, client) and IBM Granite-4.0-1B (judge), quantised to 4 bits so that both fit the 4 GB laptop GPU (GTX 1650). Model commits are pinned.
3. **Offline templates and rules**: the template advisor, the rule-based customer and the checklist.

We moved from local to hosted models after measuring them: a local advisor call took 16 s at the median because the 4 GB GPU overflowed into shared memory, against about 2 s on OpenRouter (Section 5.7). All calls are cached (SQLite) and traced (one JSON line per call: model, tokens, latency, tools, cost).

4 Experiments

4.1 Principles

Every experiment runs from one command (`callverse evaluate`) and writes its tables to `artifacts/evaluations/`, with a summary run in MLflow [27]. Four principles apply throughout:

- **A baseline and a backup for every idea**, and a switch rule fixed *before* the results are known (Table 1). The scorecard (`scorecard.json`) applies these rules automatically.
- **Common random numbers** [14]: every option is simulated on exactly the same random customers (arrivals, handle times, patience), so a difference comes from the decision only.
- **Paired differences with 95% Student-t confidence intervals** over seeds; a difference is called *significant* only when its interval excludes zero.
- **No leakage**: models are trained on the past only (rolling origins for the forecast, a training period before a test period for routing).

4.2 Experiments and protocols

Table 4 lists the experiments. Simulated weeks use 10 seeds; the routing study on stressed days uses 20 unseen days with 2 seeds each (40 pairs).

4.3 Metrics

Service. *ASA* (average speed of answer) is the mean wait of answered customers; the *mean wait of all live customers* also counts those who leave. The *service level* is the share answered within 20 s; the *abandonment rate* the share who leave before an answer; the *90th-percentile wait* describes the long waits that the mean hides. An *understaffed hour* is an open hour with at least 3 live contacts whose service level is under the 80% target. **Cost.** Labour and overtime cost, cost per handled contact, and labour plus lost-customer cost (€8 per abandonment). **Forecast.** wMAPE (Section 3) and RMSE. **Retrieval.** Recall@k, the share of questions whose correct section is among the first k results, and MRR, the mean of 1/rank of the correct section. **Answers.** The *verified policy-correct action* (the advisor's action equals the rules' action for the true intent); *faithfulness*, the share of an answer's sentences supported by its evidence (a sentence counts as supported when its best bge-m3 cosine with a sentence of a tool result or cited policy passes a threshold; a local, cheaper stand-in for an LLM-graded RAGAS [7]); similarity to the approved answer; CSAT (the simulated customer's 1–5 score) and first-contact resolution (FCR). **Judge.** Quadratic weighted Cohen's kappa [6] between the judge's 1–5 scores and the labels, "trusted" from 0.4 upwards; it corrects for chance agreement and penalises large disagreements more than small ones.

Table 4: Experiments, their question and their design.

Experiment	Question	Design
Forecast backtest	Does LightGBM beat "same day last week"?	46 weekly origins (Sundays, Oct 2017–Aug 2018), 7-day horizon, 5 models, each trained only on data before its origin
Twin fidelity	Does the twin behave like queueing theory and like the history?	Erlang C check (5 replications × 400 h); KS tests and total variation distance on 4 weeks
BO4: testing first	Does testing a plan before a peak help?	Unprepared plan (built from the normal forecast) vs prepared plan (from the scenario's forecast), 4 peak weeks × 10 seeds, FIFO routing
DSO3: planning	Is CP-SAT cheaper than a fixed schedule at the service target?	4 planners × 2 weeks (normal, promotion) × 10 seeds, FIFO routing
DSO1: routing	Do the learned and tuned routers beat FIFO?	(a) 20 unseen stressed days (Apr–Aug 2018, load ×1.6, 15% absent) × 2 seeds; (b) 3 peak weeks × 10 seeds with the unprepared plan
DSO2: retrieval	Does bge-m3 beat keyword search?	255 Bitext questions (15 per intent), 58 sections
DSO2: conversations	How good are the advisor variants?	The same 12 sampled customers answered by each variant (paired); the template variant also on 40
DSO2: judge	Can the automatic judge be trusted?	Judge scores vs labels on 60 historical chats; quadratic kappa and Spearman correlation

4.4 Monitoring the AI components

Offline experiments are complemented by a monitoring page in the back office, computed from what the system already records: the trace of every LLM call, the saved chats (live and evaluation) and the result tables. It reports, per agent and model, call volume, errors, cache hits, median (p50) and 95th-percentile (p95) latency and tokens per second; tokens and cost, with an "if we paid" estimate at a reference list price (gpt-4o-mini, \$0.15/\$0.60 per million input/output tokens), clearly labelled as an estimate since the real cost is zero; the RAG citations; tool calls, with *who chose them* (the LLM, a fixed pipeline step or a safety net); the quality scores; routing accuracy by confidence; and the agreement between the judge and the customer. Six indicators turn amber or red past thresholds kept in `configs/monitoring.yaml` (for example, p95 LLM latency: warn above 10 s, alert above 20 s; routing accuracy: warn below 85%).

5 Results

5.1 DSO4: the forecast beats the naive baseline

LightGBM reduces the error on the daily total from 31.6% (same day last week) to **25.3%** wMAPE, a 19.9% relative improvement (RMSE 7.0 contacts per day), and it is also better than SARIMA (27.7%) and Prophet (35.9%) on the total (Figure 2). Per family (Table 5), it wins on the largest one (where is my order); SARIMA is slightly better on two small families and Prophet on one. Small families have 1–3 contacts a day, so their error is dominated by Poisson noise; staffing is therefore decided on the total. The most important features are the recent levels (28- and 7-day means, 31% and 29% of the gain), the same-weekday mean (9%), the days since the last major sale and the orders promised for the coming days: the model uses the *order drivers*, as intended. **Verdict: main**

idea kept.

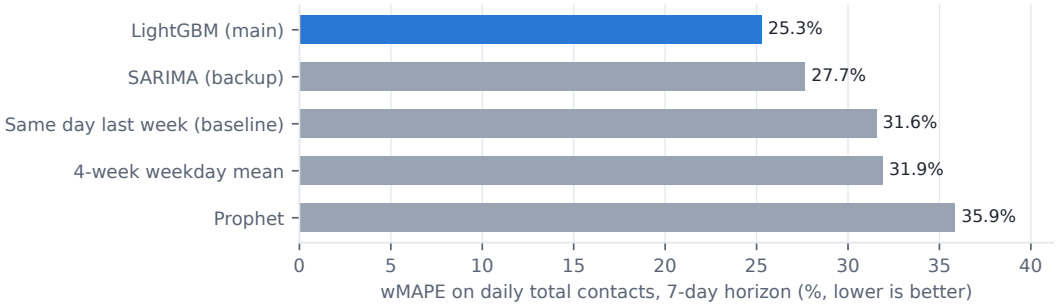


Figure 2: Forecast error on the daily total (rolling backtest, 46 origins, 7-day horizon).

Table 5: wMAPE (%) per intent family; lower is better, best in bold.

Family	LightGBM	Same day last week	4-week mean	SARIMA	Prophet
Total	**25.3**	31.6	31.9	27.7	35.9
Where is my order	**36.9**	45.5	48.8	42.3	52.1
After delivery	38.0	45.1	41.9	**35.2**	43.7
Order admin	54.7	68.0	55.9	53.7	**52.7**
Cancel / refund	83.8	102.9	86.1	**81.3**	90.5

5.2 DSO4: the twin is faithful

The twin passes all six fidelity checks (Table 6). The first three prove that the engine is correct (it reproduces Erlang C within 3%); the last three show it is consistent with its inputs: daily volumes, hourly profile and handle times are statistically indistinguishable from the history. **Verdict: kept.**

Table 6: Fidelity of the twin.

Check	Twin	Reference	Rule
Erlang C: probability of waiting	0.279	0.285 (error 2.2%)	error $< 5\%$
Erlang C: mean wait	33.3 s	34.2 s (error 2.7%)	error $< 5\%$
Erlang C: answered within 20 s	76.5%	75.9% (error 0.8%)	error $< 5\%$
Daily volume vs history (4 weeks)	491 / day	488 / day, KS $p = 0.998$	$p > 0.05$
Hour-of-day profile	—	total variation distance 0.015	< 0.05
Handle time vs ticket logs	273 s	272 s, KS $p = 0.35$	$p > 0.05$

5.3 BO4: testing a plan before a peak

For each peak week we compare the plan a manager would build from the normal forecast (*unprepared*) with the plan built after testing the scenario in the twin (*prepared*), on the same customers. Preparing always protects the service, and every difference is significant (Figure 3, Table 7): understaffed hours fall by 60–84%, abandonment by 55–78%, and the mean wait of live customers by 60–82%. Preparing costs extra labour, so the money result depends on what a lost customer is worth. With our assumption of €8, testing first saves **€6,991 net over the four peak weeks**; it loses €692 in the staff shortage week, where the break-even value of a lost customer is €12. A repeat contact plus the risk of losing the customer usually costs more than €6–12, which is why the break-even, not the euro figure, is the robust result. Running 10 replications of a week takes 4.5–15.6 seconds, so a manager can test many options.

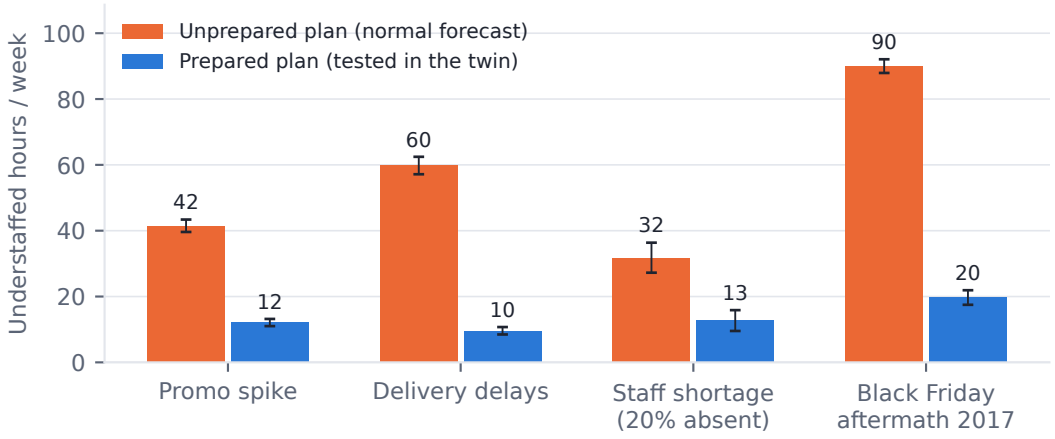


Figure 3: Understaffed hours per peak week, unprepared vs prepared plan (mean of 10 seeds, 95% CI).

Table 7: Testing first, per peak week (unprepared → prepared, 10 seeds). Net saving = avoided lost-customer cost minus extra labour, with its 95% CI.

Scenario	Answered < 20 s	Abandonment	Net saving (€)	Break-even
Promo spike	76.1% → 91.9%	11.7% → 4.5%	+226 [129; 323]	€7.4
Delivery delays	67.8% → 93.7%	15.5% → 3.8%	+1,196 [935; 1,458]	€6.2
Staff shortage (20% absent)	81.6% → 92.6%	9.8% → 4.4%	−692 [−987; −397]	€12.1
Black Friday aftermath 2017	45.0% → 87.2%	29.0% → 6.4%	+6,261 [5,985; 6,538]	€3.0

5.4 DSO3: CP-SAT planning

In a normal week the CP-SAT plan costs **€7,050**, against €11,760 for a schedule sized for the busiest hour (**−40%**), and still answers 92.4% of chats within 20 s, above the 80% target, with every labour rule respected (Figure 4). The cost per handled contact falls from €3.20 to €1.95. The rule-based backup (Erlang C with two shift templates) saves 17%. In the promotion week, CP-SAT costs €9,762 against €16,464 (**−41%**) at 91.9% service. **Verdict: main idea kept** (≥5% cheaper at the target). A fixed schedule *tuned by hand* to the right level costs about the same in a normal week, because the Olist hourly profile is flat from 10:00 to 22:00; CP-SAT's value is that it finds this level automatically for every week and scenario, guarantees the requirement hour by hour and the labour rules, and adapts when demand has peaks.

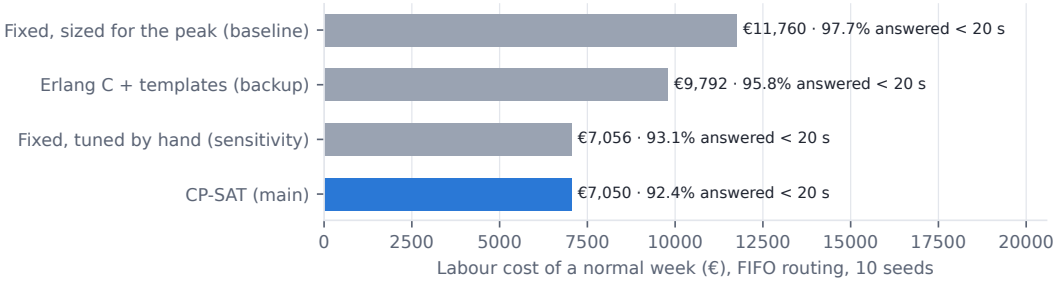


Figure 4: Weekly labour cost of the four planners in a normal week, with the service level each one reaches.

5.5 DSO1: routing

On **unseen stressed days**, both routers beat FIFO significantly (Table 8): PPO cuts the speed of answer by **13.9%** and raises the service level by 3.4 points; the tuned skill score cuts the mean wait of live customers by 11.6% and abandonment by 7.2%. In the **full peak weeks** the picture changes (Figure 5): neither router changes the mean wait significantly; both reduce abandonment significantly (skill score −6.5% to −9.6%, PPO −3.7% to −4.5%), and PPO cuts the 90th-percentile wait by 18–25%. PPO serves customers who have waited long or are about to leave: a fairness gain that the mean hides. Peak weeks differ from the training days (a backlog that carries over several days, planned overtime), and PPO was trained on single days. **Verdict (switch rule): not met**: the ≥10% gain does not hold in peak weeks. The tuned skill score is the production candidate (robust, explainable), and PPO stays the research track.

Table 8: Routing on 20 unseen stressed days (40 paired runs). In brackets: change vs FIFO; all significant.

Router	ASA	Mean wait (all live)	< 20 s	Abandonment
FIFO + skill matching (baseline)	25.2 s	37.5 s	59.7%	19.7%
Tuned skill score (backup)	22.6 s (−10.3%)	33.1 s (−11.6%)	62.2%	18.3% (−7.2%)
MaskablePPO (main)	21.7 s (−13.9%)	35.4 s (−5.6%)	63.0%	18.8% (−4.6%)

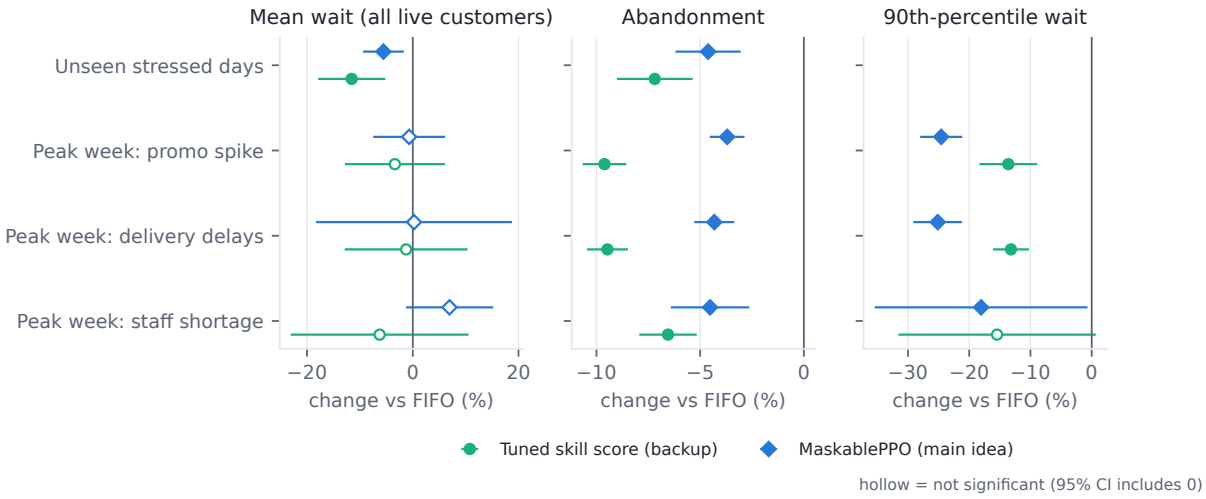


Figure 5: Change vs FIFO with 95% CI. Filled markers are significant; hollow ones are not (the 90th-percentile wait was not measured in the stressed-day study).

5.6 DSO2: retrieval, answers and the judge

Retrieval. On 255 questions, bge-m3 + FAISS finds the right policy section first 80.0% of the time and in the top 4 **96.9%** of the time, against 52.9% and 80.4% for keyword search (Figure 6). The gap is largest in French (34 questions: 94% vs 15% at rank 1), because the embeddings are multilingual. **Verdict: retrieval beats the baseline.**

Answers. Table 9 compares the variants on the same 12 customers. This paired batch was run on 19–20 October with our first local models (Qwen2.5-3B advisor and client, Llama-3.2-3B judge); the template variant was later re-run on 40 customers. The pure RAG advisor writes well-grounded answers (faithfulness 0.99), but nothing in them can be *verified* as the policy action, and the judge accepted none of them. The hybrid, where the rules engine decides and the LLM writes, gives a checkable action that is right in **12/12** chats and the best customer satisfaction (**4.33/5**); in 4 of

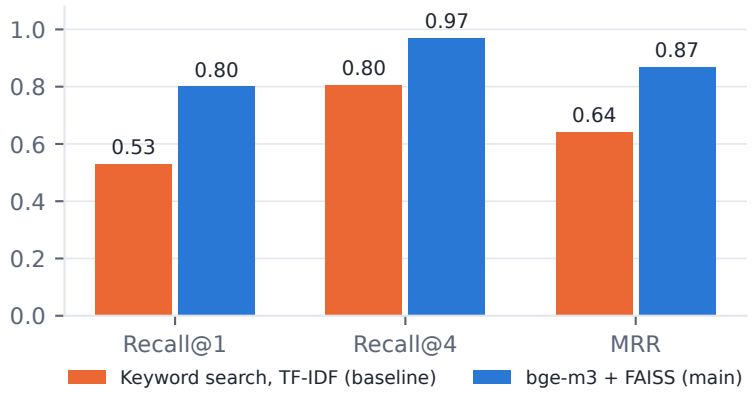


Figure 6: Retrieval on 255 Bitext questions (17 intents).

the 12 chats a safety net replaced an LLM draft by the approved text. Tool calls worked 100% of the time once tool gating was in place (48% before).

Table 9: Advisor variants (BO2 KPIs). "Verified" compares the advisor's action with the rules' action; the template variant is judged by the checklist, the LLM variants by the LLM judge.

Variant	Chats	Verified	Judge: correct	CSAT	Faithful.	Similarity
Template (backup)	40	100%	100%	3.83	0.65	1.00
Pure RAG (idea A)	12	not verifiable	0%	3.58	0.99	0.74
RAG + rules engine (hybrid)	12	**100%**	17%	**4.33**	0.70	0.79

The judge. Against labels on 60 historical chats, neither judge reaches the 0.4 kappa needed to be trusted (Table 10). The 3B LLM judge is close to chance; the checklist is better on resolution (kappa 0.39, Spearman 0.50). Two reasons: a 3B judge is too weak (it called policy-correct answers "invented promises"), and the labels are proxies (a generator's answer key), so the satisfaction labels are partly random. **Verdict: the AI reviewer is not trusted yet**; the deterministic action check is the production quality signal, and the judge is advisory.

Table 10: Judge calibration: quadratic weighted kappa on 60 chats (trusted from 0.4).

Judge	Satisfaction	Resolution	Within ±1 point
LLM judge (Llama-3.2-3B)	0.06	0.06	58–65%
Checklist reviewer (backup)	0.07	**0.39**	63–65%

Tool calling. Making the LLM choose tools taught us more than any score. SmolLM3-3B never emitted a real tool call on its own (0 of 186 traced advisor calls); it even *narrated* fake calls ("I've called lookup_order...") with invented facts. Forcing a tool call at the investigation step (the reply is pre-started with the tool-call marker) gave 6 of 6 real calls with the right order id. The hosted Nemotron models call tools correctly in about 1.5 s. On the client side, the customer's `view_my_order_page` tool caught a wrong "delivered" claim by the advisor in a trial.

5.7 Monitoring the AI components

Over the 1,339 traced LLM calls and 219 saved chats (live and evaluation, all periods), the monitoring page (Figure 7) gives a sober picture (Figures 8 and 9):

- **Latency.** The p95 latency is 28 s (red), but it comes from the local-GPU period: the local SmolLM3 advisor answers in 16.2 s at the median (p95 43.1 s), the hosted Nemotron 3.5 Super

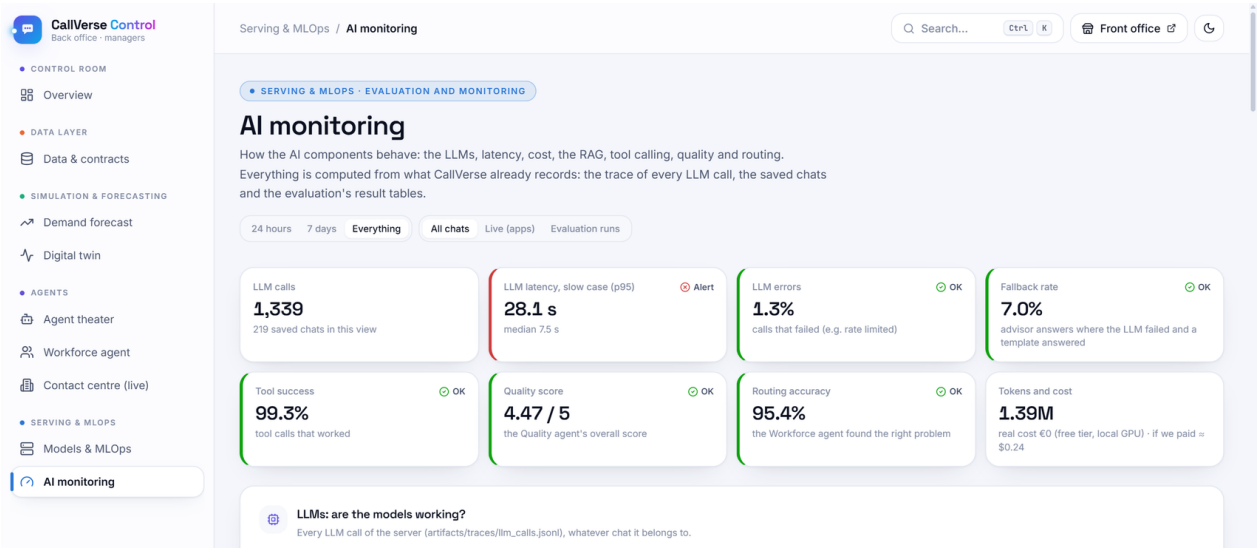


Figure 7: The AI monitoring page of the back office (all periods, all chats). Each headline indicator turns amber or red past its threshold, always with an icon and a word (OK, Watch, Alert) as well as a colour.

advisor in 2.0 s (p95 8.0 s). This measurement drove the switch to OpenRouter.

- **Fallbacks and tools.** 7.0% of the advisor's LLM answers fell back to a template (amber threshold: 20%), and 99.3% of the tool calls succeeded. Of the tool calls whose caller is recorded, 132 were made by fixed pipeline steps and only 13 were chosen by the LLM: the system is honest "workflow plus agent", not a fully autonomous agent.
- **Routing.** The Workforce agent found the right problem in 95.4% of the chats; its confidence can be trusted: 99.5% right when it is above 0.9 (186 chats), 78% between 0.5 and 0.7. The main confusion is `track_order` vs `track_delivery` (6 chats), two close intents of the same family (where is my order).
- **Judge vs customer.** The judge's score and the customer's CSAT correlate only weakly (Pearson 0.21), another sign that the judge is not yet a reliable proxy.
- **Cost.** The real cost is €0 (free tier and local GPU). At the reference price, the 1.39 million tokens would cost $0.24, about $0.0002 per resolved chat; 90.9% of the chats were resolved.

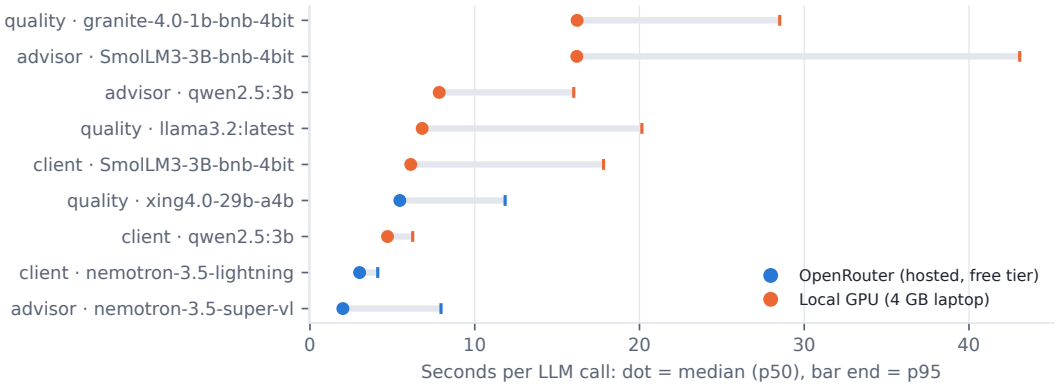


Figure 8: Latency per LLM call by agent and model (models with at least 10 calls).

5.8 Scorecard

Table 11 applies the switch rules. The foundation (DSO4) and the planning objective (DSO3) pass clearly. DSO1 and DSO2 bring measurable gains, but our own rules put the backup or a hybrid in

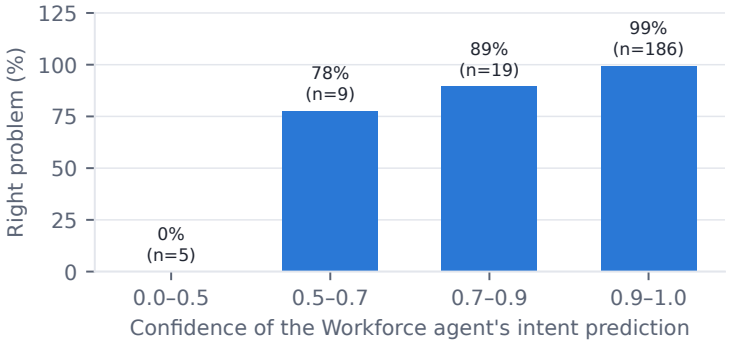


Figure 9: Routing accuracy by confidence of the intent prediction (219 chats).

production.

Table 11: BO/DSO scorecard (`artifacts/evaluations/scorecard.json`).

DSO	Result	Rule met	In production
DSO1 routing	PPO: ASA −13.9% on stressed days; in peak weeks mean wait not significant, abandonment −4%	no	Tuned skill score
DSO2 advisor + judge	Recall@4 96.9%; hybrid 12/12 verified actions, CSAT 4.33; judge kappa < 0.4	no	Hybrid advisor; template backup; checklist reviewer
DSO3 planning	Labour cost −40% at 92.4% service; labour rules respected	yes	CP-SAT
DSO4 forecast + twin	wMAPE 25.3% vs 31.6%; all fidelity checks pass; €6,991 net saved over 4 peak weeks	yes	LightGBM + twin

6 Critical analysis and limitations

What the evidence supports. The strongest results are the ones with the most data and the simplest claims: the forecast beats its baseline over 46 origins, the engine reproduces queueing theory, and planning and testing first show large, significant effects over 10 seeds. Where the AI ideas did not meet their own rule (routing in peak weeks, the judge), the project shipped the backup or a hybrid instead of hiding the result. We see this discipline (a baseline, a confidence interval and a fallback for every claim) as the main methodological result.

1. Synthetic contacts. Contacts, tickets and chats are generated from real orders with assumed rates, handle times and patience. The twin's agreement with the "history" is therefore partly circular: it shows consistency with its inputs, not with a real centre. The euro amounts depend on our cost assumptions (€12 per hour, €8 per lost customer); this is why we report break-even values. The pipeline accepts a client's real logs unchanged, and the fidelity checks would then become a real calibration.

2. Reinforcement learning does not transfer. PPO was trained on single busy days, but peak weeks carry a backlog over several days and use planned overtime. It wins on the conditions it was trained for and loses its mean-wait advantage elsewhere. Its policy is also hard to explain to a

planner, while the skill score has four readable weights. Training on multi-day episodes that match the scenarios is the obvious next step.

3. Small language models invent facts. The 3B models invented order ids, narrated tool calls they never made, and the simulated customer sometimes swapped roles with the advisor. Our answer was architectural: *the rules engine decides, the LLM writes*, tool gating, and safety nets. This works (12/12 verified actions), but it also means that the system is mostly a workflow: only 13 of the 145 tool calls with a recorded caller were chosen by the LLM. Larger hosted models call tools correctly, so autonomy can grow as models improve, but every new autonomous step needs its own measurement.

4. The automatic judge is not trusted. Kappa stays below 0.4 for both judges, and the judge's score correlates only weakly with the customer's (0.21). Part of the problem is the judge (a 3B model), part is the labels (proxies from the generator). The calibration code switches to real human grades as soon as 20 exist (target 60); collecting them is the main open item, and until then the deterministic action check is the quality signal.

5. The DSO2 sample is small and dated. The paired conversation batch covers 12 customers, too few for tight intervals, and it was run with our first local models (Qwen2.5-3B, Llama-3.2-3B), which the project no longer uses. Re-running it on the hosted models is limited by the free tier (50 requests per day, while one conversation takes a few dozen calls). The monitoring numbers cover the current models, but not as a paired experiment.

6. Dependence on a free hosted service. OpenRouter's free models are fast and cost nothing, but they are rate-limited, their availability varies, and, unlike local models, the conversation text leaves the machine. Our three-tier design (hosted, local, templates) keeps the twin working when the quota runs out, at the price of simpler answers.

7. Scope of the twin. One channel (chat), one queue per family, four problem families and Brazilian e-commerce only. Real centres mix voice, e-mail and chat and route by language and seniority. The phone page of the application is a live demonstration, not a channel modelled in the simulation.

8. Metrics have limits too. Our faithfulness measure is an embedding similarity, a cheap local stand-in for an LLM-graded RAGAS; it scores templates low only because formatted dates do not match raw JSON fields. In a normal week, a hand-tuned fixed schedule costs as little as CP-SAT, because the Olist hourly profile is flat: CP-SAT's advantage is automation and guarantees, not a lower cost in every week.

What we would do differently. Collect a small set of real, human-labelled chats early, to calibrate the judge before building on it; decide the LLM provider earlier, since the move from local to hosted models came late and left the DSO2 batch on old models; and train the RL router on the same multi-day episodes that the evaluation uses.

7 Deployment and reproducibility

7.1 The application

One FastAPI [21] backend serves two React applications (Figure 11):

- the **front office**, for customers and advisors: a help centre with semantic search, demo customers on real Olist orders, a live chat with the AI advisor (streamed word by word, with a "behind the scenes" trace of every agent, tool and policy section, then a satisfaction survey and the quality review), an advisor desk (queue, copilot drafts, autopilot), voice input through the browser's speech recognition, and a phone page;
- the **back office**, for managers: data and contracts, the forecast, the digital twin (a live floor that replays a simulated day contact by contact, and a scenario lab), the agent theatre, workforce planning, models and MLOps, the **AI monitoring** page (Section 5.7), and a full-screen **live contact centre** (Figure 10) where every move of the animated staff is a real event: a customer walks in (an AI customer, an existing customer or a new one with a new problem), the receptionist (Workforce agent) routes the chat, the advisor answers with tool badges in its speech bubbles, and the quality lead posts the score. Real chats from the portal or the phone appear there too, and each person can speak their lines aloud.

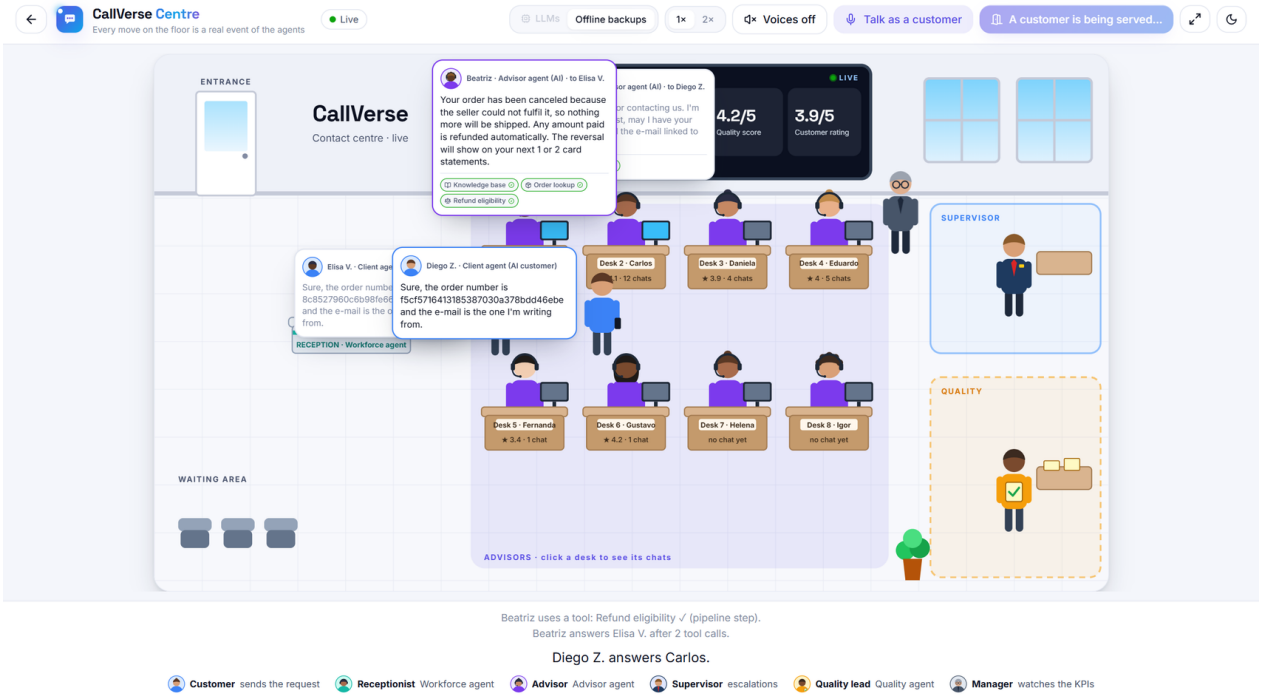


Figure 10: The live contact centre (back office, full screen), here with the offline templates. Two AI customers were routed by the Workforce agent to desks 1 and 2. Beatriz's answer carries the badges of the tools behind it (knowledge base, order lookup, refund eligibility); the footer narrates each real event, such as "Beatriz uses a tool: Refund eligibility (pipeline step)".

The live chat and the contact centre use WebSockets. The API is documented with OpenAPI at `/docs`, and an MCP server exposes the business tools to outside AI applications (stdio or `/mcp` over HTTP).

7.2 One container

The whole application ships as **one Docker image**. A first stage builds the two React apps with Node; a second stage installs the Python package on Python 3.12 and serves both apps from the API: the front office at `/`, the back office at `/back-office/`, the API at `/api`, `/ws`, `/docs` and `/mcp`. Three details make the image reproducible: the library versions are pinned to those of the environment that trained the saved models (`space/constraints.txt`, 31 libraries, so the scikit-learn classifier, the PPO policy and the MLflow database load unchanged); the bge-m3 model is downloaded at build time and the build fails if it cannot load *offline*; and the Linux builds of the JavaScript build tools are added, because the lock file was produced on Windows. For local use, Docker Compose also runs

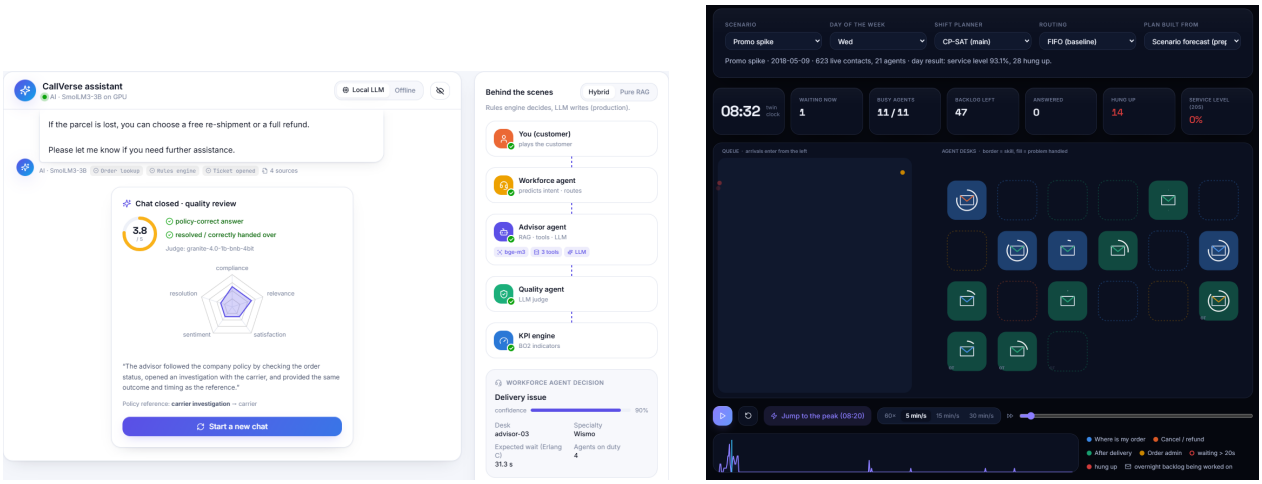


Figure 11: Left: the front office after a chat, with the quality review and the "behind the scenes" trace (captured on 1 November 2026, in the local-model period). Right: the twin's live floor replaying a promotion day with a CP-SAT plan.

MLflow with PostgreSQL and an nginx front.

7.3 Online deployment

Our first target was Hugging Face Spaces, which used to host Docker apps for free. On 11 November 2026 the platform answered that Docker Spaces now require a paid subscription (HTTP 402), so we deploy on **Azure for Students**. Its credit-only subscriptions cannot run cloud image builds (ACR Tasks), and pushing a 6 GB image over our connection would take hours. We therefore use a small Linux virtual machine (Ubuntu 24.04, 2 vCPUs, 8 GB): `scripts/deploy_azure.py` sends only a cleaned build folder (about 120 MB compressed), the VM downloads the heavy parts and builds the image itself, and a Caddy container in front obtains a free Let's Encrypt certificate. HTTPS is required for the microphone and for secure WebSockets. The demo runs at [public address, added once the Azure VM is online].

Before anything leaves the laptop, the deployment script copies an *allowlist* of files (never `.env`, keys or caches), deletes the customers we created while testing (our own names) from the copied database and compacts it, so the deleted rows cannot be recovered, leaves out our live chat transcripts, keeps only the numbers of the LLM trace, and stops if it finds a secret or one of the removed names. The OpenRouter key goes to the VM through SSH input into a file only the service user can read. The public demo has no login, and visitor data is reset at each deployment: it is a demonstration, not a production service.

7.4 Reproducibility and code quality

- **One command per stage**: `callverse all` runs the whole pipeline (data, contracts, forecast, RL training, experiments, scorecard); `callverse evaluate` re-runs every experiment of Section 4. Notebooks are executed headless and their outputs archived.
- **Fixed randomness**: seeds live in `configs/`; experiments use common random numbers; CP-SAT runs on one thread so that the roster does not change between runs.
- **Pinned models and versions**: Hugging Face commits are recorded (`llm_models.lock.json`), LLM answers are cached, and library versions are pinned for deployment.
- **Tracking**: MLflow records the parameters, metrics and artefacts of the forecast, RL and evaluation runs; every LLM call is traced.
- **Tests and checks**: 200 automated tests (unit, smoke and integration; they cover every layer, including live WebSocket sessions, the contact-centre hub and the serving of the built apps), the

`ruff` linter, six import contracts for the layered architecture, and a GitHub Actions workflow that runs them and type-checks and builds both web apps.
- **Documentation**: the README (installation, execution, structure), a step-by-step run guide, the data reports of every phase, and the experiment protocol.

8 Conclusion and future work

CallVerse shows that forecasting, discrete-event simulation, optimisation, reinforcement learning and language-model agents can work together on one business problem: running a customer service centre well. The foundation clearly beats its baselines: the forecast lowers the error from 31.6% to 25.3%, and the twin reproduces queueing theory and the history. On top of it, CP-SAT planning cuts labour cost by 40% at the service target, and testing a plan before a peak cuts understaffed hours by 60–84%. Routing and AI answering bring measurable gains, but our own switch rules sent a tuned rule and a hybrid advisor (the rules engine decides, the LLM writes) to production, while the pure-AI versions remain research tracks. The automatic judge is not yet trusted, and we say so. The application, from the customer portal to the live contact-centre view and the monitoring of every AI component, is tested, documented and packaged as one container, ready for online deployment.

Future work.

- **Real data**: calibrate contact rates, handle times and patience on a partner's logs, and turn the fidelity checks into a real calibration.
- **Human labels**: grade 60 real conversations to calibrate the judge, then try a stronger hosted judge from another model family.
- **Routing**: train PPO on multi-day episodes matching the peak scenarios, and compare it with the skill score behind an A/B test.
- **Agents**: re-run the paired DSO2 batch on the hosted models with a larger sample, and widen the LLM's own tool choices step by step, each with its measurement.
- **Monitoring**: add a chat inspector timeline, drift detection on the incoming intents, a safety test set, and a one-click regression run before each deployment.
- **Product**: more channels in the twin (voice, e-mail), connectors to help-desk tools, and authentication for the online application.

9 Declaration of the use of generative AI tools

As required by the assessment rules, we declare the generative AI tools used in this project.

- **Claude (Anthropic), through Claude Code**, was used as a programming and writing assistant throughout the project: to write and refactor code and tests, to build the web interfaces, to write the documentation and the deployment scripts, and to draft this report from our documentation and result files. The figures of this report are drawn by a script (`report/make_figures.py`) directly from the result tables, and every number comes from those tables or from the project's reports.
- **Our own responsibility.** We chose the problem, the objectives and the switch rules, made the design decisions, ran the experiments, checked the results and reviewed every section. Every result can be reproduced with the commands of Section 7.
- **Language models inside the product** are part of the system studied, not tools used to write it: NVIDIA Nemotron 3.5 and Xing 4.0 (through OpenRouter), SmolLM3-3B and IBM Granite-4.0-1B (local), and, in the first experiments, Qwen2.5 and Llama 3.2. They are described in Section 3.7.

References

[1] Anthropic. Model Context Protocol specification. `https://modelcontextprotocol.io`. Accessed November 2026.

[2] Niels Bantilan. pandera: Statistical data validation of Pandas dataframes. In *Proceedings of the 19th Python in Science Conference (SciPy)*, pages 116–124, 2020.

[3] Bitext. Bitext retail e-commerce LLM chatbot training dataset. Hugging Face, `https://huggingface.co/datasets/bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset`. Licence CDLA-Sharing 1.0. Accessed November 2026.

[4] Pete Chapman, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rüdiger Wirth. CRISP-DM 1.0: Step-by-step data mining guide. Technical report, SPSS, 2000.

[5] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.

[6] Jacob Cohen. Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[7] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the ACL: System Demonstrations*, 2024.

[8] Noah Gans, Ger Koole, and Avishai Mandelbaum. Telephone call centers: Tutorial, review, and research prospects. *Manufacturing & Service Operations Management*, 5(2):79–141, 2003.

[9] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the 35th International FLAIRS Conference*, 2022.

[10] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3rd edition, 2021. `https://otexts.com/fpp3/`.

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[12] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, 2017.

[13] LangChain. LangGraph: building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`. Accessed November 2026.

[14] Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill, 5th edition, 2015.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, 2020.

[16] John D. C. Little. A proof for the queuing formula: L = λ W. *Operations Research*, 9(3):383–387, 1961.

[17] Olist. Brazilian e-commerce public dataset by olist. Kaggle, `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`. Licence CC BY-NC-SA 4.0, 2018.

[18] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[19] Laurent Perron and Vincent Furnon. OR-Tools (version 9.15). `https://developers.google.com/optimization`. Accessed November 2026.

[20] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[21] Sebastián Ramírez. FastAPI. `https://fastapi.tiangolo.com`. Accessed November 2026.

[22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[23] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.

[24] Team SimPy. SimPy: Discrete event simulation for Python. `https://simpy.readthedocs.io`. Accessed November 2026.

[25] Mark Towers et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Matei Zaharia et al. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018.

[28] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.